

Assignment 5: Advanced Multi-client chat server using TCP

Name : Piyush Kumar

Roll : CSB24051

1. Objective

The objective of this assignment is to enhance the TCP Multi-Client Chat Server developed in Assignment 4 by implementing advanced communication features such as private messaging, online user management, persistent chat history, server statistics, and performance evaluation.

2. System Architecture

Operating system (uname -a) : Linux msi UBUNTU

Compiler (gcc --version) : gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0

Network (ip addr) :

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
group default qlen 1000

link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00

```
inet 127.0.0.1/8 scope host lo
    valid_lft forever preferred_lft forever

inet 10.255.255.254/32 brd 10.255.255.254 scope global lo
    valid_lft forever preferred_lft forever

inet6 ::1/128 scope host proto kernel_lo
    valid_lft forever preferred_lft forever

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP
group default qlen 1000

link/ether 00:15:5d:39:22:8f brd ff:ff:ff:ff:ff:ff

altname enx00155d39228f

inet 192.168.67.131/20 brd 192.168.79.255 scope global eth0
    valid_lft forever preferred_lft forever

inet6 fe80::215:5dff:fe39:228f/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
```

3. Design Improvements over Assignment 4

Compared with Assignment 4, several advanced features were added.

Assignment 4

Broadcast chat

Basic client list

Temporary
messages

Simple server
log

No
performance
evaluation

No previous
history

No user
commands

Assignment 5

Broadcast + Private
chat

Online user
management

Persistent chat
history

Detailed client
statistics

Delay and
throughput
analysis

Last five messages
on reconnect

/list and /msg
commands

4. Program Design

Server Responsibilities

The server performs the following operations:

- Accepts multiple TCP connections
- Creates one thread for every client
- Registers usernames
- Maintains active client information
- Routes broadcast messages
- Routes private messages
- Updates chat history

Client Information Stored

Each connected client contains:

- Username
- IP Address
- Port Number
- Login Time
- Online/Offline Status

Persistent Chat History

All messages are stored in

chat_history.csv

5. Experimental Setup

Environment

Component	Value
Operating System	Ubuntu Linux
Network Emulator	Mininet
Protocol	TCP
Programming Language	Python 3
Port Number	5000
Packet Analyzer	Wireshark

Performance Experiment

Experiments were conducted using

- 2 clients
- 3 clients
- 4 clients

Each client transmitted

50 messages

Measurements collected:

- Average delivery delay
 - Throughput
 - Broadcast messages
 - Private messages
-

6. Results

Performance Results

Clients	Broadcast	private	avg_delay	throughput_msgs_per_sec
2	80	20	1.127	34.965
3	120	30	1.068	50.37
4	160	40	7.187	58.885

7. Graph Analysis

Three graphs were generated.

Graph 1

Number of Clients vs Average Delivery Time

Observation

As the number of clients increases, average message delivery time also increases due to additional server processing and increased network traffic.

Figure 1: clients_vs_delay.png



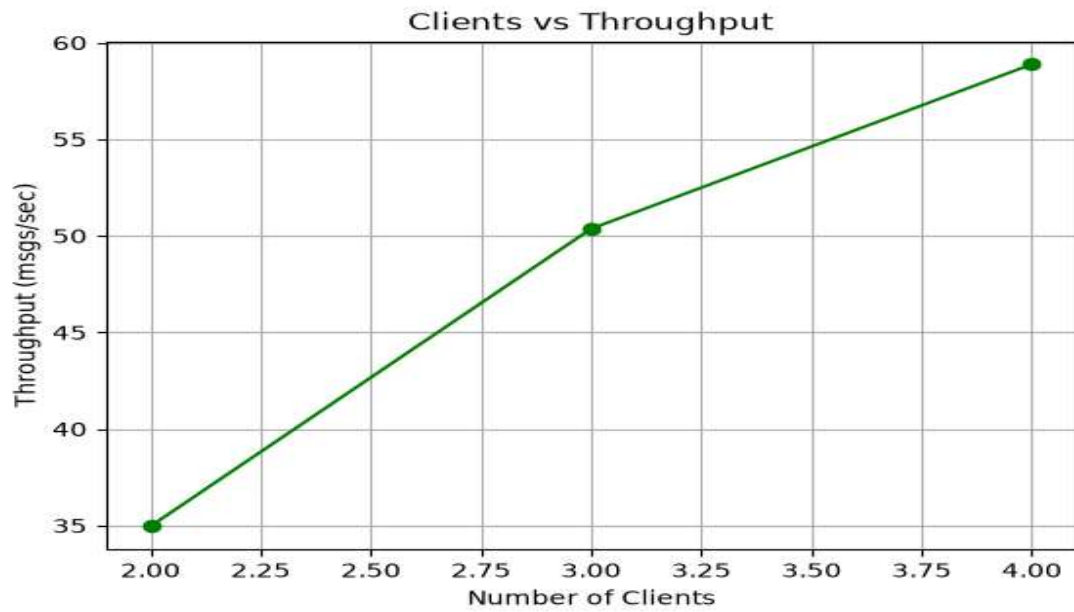
Graph 2

Number of Clients vs Throughput

Observation

Throughput decreases slightly as the number of concurrent clients increases because the server processes more simultaneous messages.

Figure 2: clients_vs_throughput.png



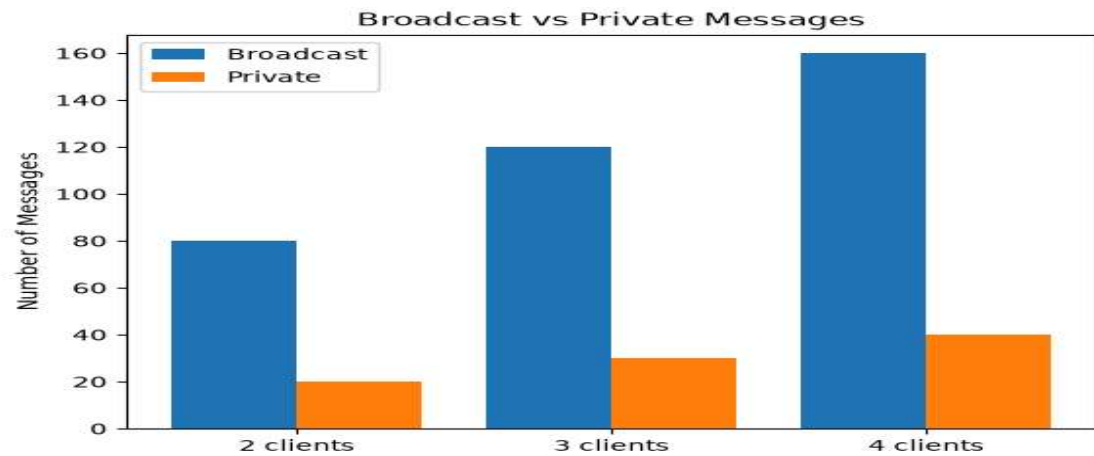
Graph 3

Broadcast Messages vs Private Messages

Observation

Broadcast messages are generally higher than private messages because broadcast communication reaches all users simultaneously.

Figure 3: message_type_distribution.png



8. Wireshark Verification

TCP traffic was captured using the filter

```
tcp.port == 5000
```

The following events were verified.

Screenshot 1

Client establishes TCP connection.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.2	10.0.0.1	TCP	74	36446 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=3641476097 TSecr=0 WS=512
2	0.000129173	10.0.0.1	10.0.0.2	TCP	74	5000 → 36446 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=4231182694 TSecr=3641476097 WS=512
3	0.000365360	10.0.0.2	10.0.0.1	TCP	66	36446 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=3641476098 TSecr=4231182694

Screenshot 2

```
message_17_from_1
[Piyush_2]: message_14_from_2
[Piyush_0]: message_18_from_0
[OK]
[Piyush_0]: message_19_from_0
```

Broadcast message packet.

Screenshot 3

Private message transmission.

```
/msg perf_user_2 hello_15
[Piyush_2]: message_12_from_2
[Piyush_0]: message_16_from_0
```

Screenshot 4

```
message_49_from_1
*** Piyush_0 has left the chat ***
[OK]
[OK]
[Piyush_2]: message_47_from_2
/quit
```

Client disconnection.

Screenshot 5

TCP connection termination using

- FIN

584	963.017669778	10.0.0.1	10.0.0.3	TCP	66 5000 → 35196	[FIN, ACK]	Seq=58 Ack=153 Win=43520 Len=0 TSval=4217660326 TSecr=3670778517
585	963.017741221	10.0.0.1	10.0.0.4	TCP	66 5000 → 46416	[FIN, ACK]	Seq=396 Ack=159 Win=43520 Len=0 TSval=231850651 TSecr=2753056240
586	963.017752065	10.0.0.1	10.0.0.2	TCP	66 5000 → 52902	[FIN, ACK]	Seq=747 Ack=159 Win=43520 Len=0 TSval=2220215398 TSecr=3292484673

9. Reflection Answers

Q1. Why is server-side routing necessary for private messaging?

The server maintains information about all connected clients. It identifies the intended recipient and forwards the message only to that client, preventing other users from receiving confidential messages.

Q2. How does maintaining user state improve the application?

Maintaining user state allows the server to track online users, manage login sessions, restore previous chat history, and provide accurate client information, improving communication reliability.

Q3. How would the application scale to 100 users?

To support 100 users, the server could use asynchronous I/O or thread pools instead of one thread per client, optimize data structures, improve message queues, and deploy load balancing if necessary.

Q4. Which Wireshark packets verified private messaging?

TCP packets containing application-layer data exchanged only between the sender and the server, followed by packets from the server to the intended recipient, confirmed successful private message routing.

Q5. What improvements would you implement next?

Possible improvements include:

- Group chat support
- File transfer
- End-to-end encryption

-
- Graphical User Interface
 - Authentication system
 - Database storage
 - Offline messaging
 - Voice and video communication
-

10. Conclusion

This assignment successfully extended the multi-client TCP chat application by implementing advanced client management, private messaging, online user listing, persistent chat history, and performance analysis. The server efficiently handled concurrent client communication while maintaining user state and routing messages correctly. Performance measurements and Wireshark packet captures verified the correctness of the implementation. The assignment provided practical experience with TCP socket programming, multithreading, client-server architecture, network traffic analysis, and software evolution.